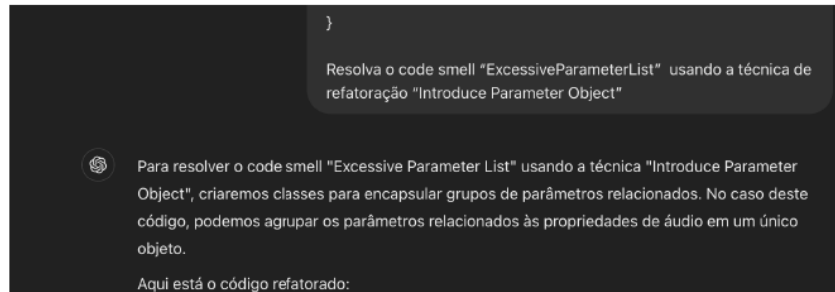


ExcessiveParameterList AudioReference editAudio

AudioReference editAudio

1. Foi enviado para a IA o código da classe AudioReference com o método editAudio com o seguinte comando:



Resultado:

```
package org.example.studymaterial;

public class AudioProperties {
    private String title;
    private String description;
    private String link;
    private String accessRights;
    private String license;
    private String language;
    private int rating;
    private int viewCount;
    private int shareCount;
    private boolean isDownloadable;

    public AudioProperties(String title, String description, String link, String accessRights, String
license, String language, int rating, int viewCount, int shareCount, boolean isDownloadable) {
        this.title = title;
        this.description = description;
        this.link = link;
        this.accessRights = accessRights;
        this.license = license;
        this.language = language;
        this.rating = rating;
        this.viewCount = viewCount;
        this.shareCount = shareCount;
        this.isDownloadable = isDownloadable;
    }

    public String getTitle() {
        return title;
    }

    public String getDescription() {
        return description;
    }

    public String getLink() {
        return link;
    }

    public String getAccessRights() {
        return accessRights;
    }
}
```

```

    public String getLicense() {
        return license;
    }

    public String getLanguage() {
        return language;
    }

    public int getRating() {
        return rating;
    }

    public int getViewCount() {
        return viewCount;
    }

    public int getShareCount() {
        return shareCount;
    }

    public boolean isDownloadable() {
        return isDownloadable;
    }
}

```

- Logo em seguida foi pedido que usasse record no lugar de uma nova classe estruturada, para que assim se evitasse alguns efeitos colaterais.


Use record ao invés de uma classe nova



Se você está usando uma versão do Java que suporta `record` (introduzido no Java 14 como preview e estabilizado no Java 16), podemos substituir a classe `AudioProperties` por um `record`. Um `record` é uma forma concisa e imutável de declarar uma classe que é usada principalmente para armazenar dados.

Aqui está a refatoração utilizando um `record`:

Refatoração com `record`

Resultado:

```

package org.example.studymaterial;

import java.util.List;

public class AudioReference extends Reference {
    public enum AudioQuality {
        LOW, MEDIUM, HIGH, VERY_HIGH;
    }

    private AudioQuality audioQuality;

    public AudioReference(AudioQuality quality) {
        this.audioQuality = quality;
    }

    public AudioQuality getAudioQuality() {
        return audioQuality;
    }

    public static AudioQuality audioQualityAdapter(String quality) {
        return switch (quality.toLowerCase()) {

```

```

        case "low" -> AudioQuality.LOW;
        case "medium" -> AudioQuality.MEDIUM;
        case "high" -> AudioQuality.HIGH;
        case "very_high" -> AudioQuality.VERY_HIGH;
        default -> null;
    };
}

public void setAudioQuality(AudioQuality audioQuality) {
    this.audioQuality = audioQuality;
}

// Record para encapsular os parâmetros de editAudio
public record AudioAttributes(
    String title,
    String description,
    String link,
    String accessRights,
    String license,
    String language,
    int rating,
    int viewCount,
    int shareCount,
    boolean isDownloadable
) {}

public void editAudio(AudioQuality audioQuality, AudioAttributes attributes) {
    editBasic(attributes.title(), attributes.description(), attributes.link());
    this.setAccessRights(attributes.accessRights());
    this.setLicense(attributes.license());
    this.setAudioQuality(audioQuality);
    editVideoAttributes(attributes);
}

public void editAudioAdapter(List<String> properties, List<Integer> intProperties, AudioQuality aud
    AudioAttributes attributes = new AudioAttributes(
        properties.get(0), // title
        properties.get(1), // description
        properties.get(2), // link
        properties.get(3), // accessRights
        properties.get(4), // license
        properties.get(5), // language
        intProperties.get(0), // rating
        intProperties.get(1), // viewCount
        intProperties.get(2), // shareCount
        isDownloadable // isDownloadable
    );
    this.editAudio(audioQuality, attributes);
}

private void editVideoAttributes(AudioAttributes attributes) {
    this.setRating(attributes.rating());
    this.setLanguage(attributes.language());
    this.setDownloadable(attributes.isDownloadable());
    this.getInteractionStatistics().setViewCount(attributes.viewCount());
    this.getInteractionStatistics().setShareCount(attributes.shareCount());
}

public void editBasic(String title, String description, String link) {
    this.setTitle(title);
    this.setDescription(description);
    this.setLink(link);
}
}

```

2. A refatoração da IA causou efeitos colaterais em outros métodos. Foi então solicitado que esses erros fossem resolvidos:

```
1 - add study item
2 - add study goal
3 - add study material (audio, video, text)
4 - add study objective
5 - add study plan
6 - set up week
7 - get week responsibilities
""");
}
}
```

Agora editAudio nessa controller está com error.

O erro ocorre porque a assinatura do método `editAudio` na classe `AudioReference` foi alterada para usar um `record (AudioAttributes)` para encapsular os parâmetros. Portanto, o método na controller que chama `editAudio` precisa ser atualizado para criar e passar um objeto `AudioAttributes` em vez de passar os parâmetros diretamente.

Aqui está a correção para o método `editAudio` na controller:

Resultados:

```
private void editAudio(AudioReference audioReference) {
    handleMethodHeader("(Audio Edit)");
    System.out.println("Type the following info: AudioQuality (LOW | MEDIUM | HIGH | VERY_HIGH), boolean
        \"String title, String description, String link, String accessRights, String license, String
        \"int viewCount, int shareCount \\n\");

    // Obter os valores necessários para criar um AudioAttributes
    AudioReference.AudioQuality quality = AudioReference.audioQualityAdapter(getInput());
    boolean isDownloadable = Boolean.parseBoolean(getInput());
    String title = getInput();
    String description = getInput();
    String link = getInput();
    String accessRights = getInput();
    String license = getInput();
    String language = getInput();
    int rating = Integer.parseInt(getInput());
    int viewCount = Integer.parseInt(getInput());
    int shareCount = Integer.parseInt(getInput());

    // Criar um AudioAttributes com os valores coletados
    AudioReference.AudioAttributes attributes = new AudioReference.AudioAttributes(
        title, description, link, accessRights, license, language, rating, viewCount, shareCount, i
    );

    // Chamar o método editAudio passando o AudioQuality e o AudioAttributes
    audioReference.editAudio(quality, attributes);
}
```

3. Por fim, foi solicitado que um code smell resultante das alterações fosse resolvido:

Essa modificação causou um codesmell de Long Method. Resolva este também.

Para resolver o `code smell` de "Long Method" no método `editAudio`, podemos aplicar a técnica de `Extract Method`, que consiste em dividir partes do método em métodos auxiliares menores e mais específicos. Isso melhora a legibilidade e mantém o código organizado.

Resultado:

```
private void editAudio(AudioReference audioReference) {
    handleMethodHeader("(Audio Edit)");
    AudioReference.AudioQuality quality = promptAudioQuality();
    AudioReference.AudioAttributes attributes = promptAudioAttributes();
```

```

        audioReference.editAudio(quality, attributes);
    }

    private AudioReference.AudioQuality promptAudioQuality() {
        System.out.println("Type the AudioQuality (LOW | MEDIUM | HIGH | VERY_HIGH): ");
        return AudioReference.audioQualityAdapter(getInput());
    }

    private AudioReference.AudioAttributes promptAudioAttributes() {
        System.out.println("Type the following info: boolean isDownloadable, String title, String description,
            String link, String accessRights, String license, String language, int rating, int viewCount, int shareCount");

        boolean isDownloadable = Boolean.parseBoolean(getInput());
        String title = getInput();
        String description = getInput();
        String link = getInput();
        String accessRights = getInput();
        String license = getInput();
        String language = getInput();
        int rating = Integer.parseInt(getInput());
        int viewCount = Integer.parseInt(getInput());
        int shareCount = Integer.parseInt(getInput());

        return new AudioReference.AudioAttributes(
            title, description, link, accessRights, license, language, rating, viewCount, shareCount, isDownloadable
        );
    }
}

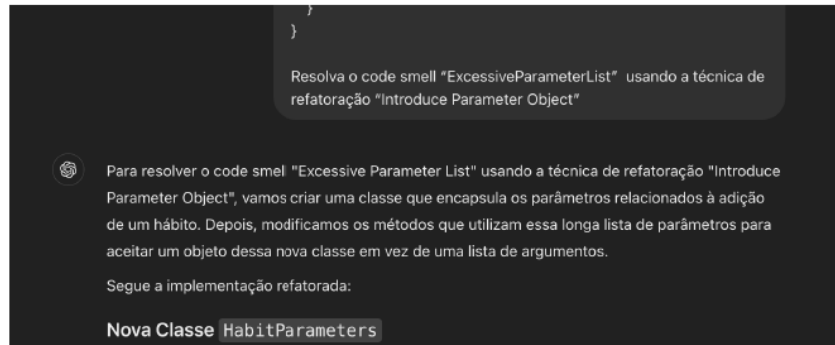
```

3. Código livre do code smell e funcionando

ExcessiveParameterList HabitTracker addHabit

HabitTracker addHabit

1. Foi enviado para a IA o código da classe HabitTracker com o método addHabit com o seguinte comando:



Resultado:

```
package org.example.studyplanner;  
  
import java.time.LocalDateTime;  
import java.time.LocalTime;  
  
public class HabitParameters {  
    private String name;  
    private String motivation;  
    private Integer dailyMinutesDedication;  
    private Integer dailyHoursDedication;  
    private LocalDateTime startDate;  
    private Boolean isConcluded;  
  
    public HabitParameters(String name, String motivation, Integer dailyMinutesDedication,  
                           Integer dailyHoursDedication, LocalDateTime startDate, Boolean isConclude  
d) {  
        this.name = name;  
        this.motivation = motivation;  
        this.dailyMinutesDedication = dailyMinutesDedication;  
        this.dailyHoursDedication = dailyHoursDedication;  
        this.startDate = startDate;  
        this.isConcluded = isConcluded;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getMotivation() {  
        return motivation;  
    }  
  
    public Integer getDailyMinutesDedication() {  
        return dailyMinutesDedication;  
    }  
  
    public Integer getDailyHoursDedication() {  
        return dailyHoursDedication;  
    }  
  
    public LocalDateTime getStartDate() {  
        return startDate;  
    }  
}
```

```

    }

    public Boolean getIsConcluded() {
        return isConcluded;
    }
}

```

2. Logo em seguida foi pedido que usasse record no lugar de uma nova classe estruturada, para que assim se evitasse alguns efeitos colaterais.

Use record ao invés de uma classe nova

Se você estiver usando uma versão do Java que suporta **records** (introduzido no Java 14 como um recurso de prévia e finalizado no Java 16), podemos simplificar ainda mais o código. Um record é uma classe imutável e concisa que encapsula os dados necessários. Substituímos a classe `HabitParameters` por um **record**, o que torna o código mais limpo e direto.

Substituindo por `record`

Aqui está o `record` que substitui a classe:

```

java
Copy code

package org.example.studyplanner;

import java.time.LocalDateTime;

public record HabitParameters(
    String name,
    String motivation,
    Integer dailyMinutesDedication,
    Integer dailyHoursDedication,
    LocalDateTime startDate,
    Boolean isConcluded
) {}

```

Resultado:

```

package org.example.studyplanner;

import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class HabitTracker {
    private List<Habit> habits;
    private Map<Integer, List<LocalDateTime>> tracker;
    private final Map<Habit, List<ToDo>> habitToDos = new HashMap<>();
    private Integer nextId;

    private static HabitTracker instance;

    public static HabitTracker getHabitTracker() {
        if (instance == null) {
            instance = new HabitTracker();
        }
        return instance;
    }

    private HabitTracker() {
        this.habits = new ArrayList<>();
        this.tracker = new HashMap<>();
        this.nextId = 1;
    }
}

```

```

@Override
public String toString() {
    StringBuilder response = new StringBuilder();
    for (Habit habit : habits) {
        response.append(habit.toString()).append(", ");
    }
    return "Habits: " + response.toString();
}

public Habit getHabitById(Integer id) {
    return this.habits.stream()
        .filter(habit -> Objects.equals(habit.getId(), id))
        .findFirst().orElse(null);
}

public void assignToHabit(ToDo toDo, Habit habit) {
    habitToDos.computeIfAbsent(habit, k -> new ArrayList<>()).add(toDo);
    System.out.println("ToDo '" + toDo.getTitle() + "' assigned to Habit: " + habit.getName());
}

public List<ToDo> getToDosForHabit(Habit habit) {
    return habitToDos.getOrDefault(habit, new ArrayList<>());
}

public List<Habit> getHabits() {
    return this.habits;
}

public String formatHabitDate(LocalDate date) {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
    return date.format(formatter);
}

public List<Integer> getTrackerKeys() {
    return this.tracker.keySet().stream().toList();
}

public int addHabit(HabitParameters params) {
    LocalTime lt = LocalTime.of(params.dailyHoursDedication(), params.dailyMinutesDedication());
    Habit habit = new Habit(params.name(), params.motivation(), lt, this.nextId, params.startDate());
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}

public int handleAddHabitAdapter(List<String> stringProperties, List<Integer> intProperties, boolean
    LocalDate startDate = LocalDateTime.of(intProperties.get(2), intProperties.get(3), intProperties.get(5), intProperties.get(6), intProperties.get(7));
    HabitParameters params = new HabitParameters(
        stringProperties.get(0),
        stringProperties.get(1),
        intProperties.get(0),
        intProperties.get(1),
        startDate,
        isConcluded
    );
    return addHabit(params);
}

public int addHabit(String name, String motivation) {
    Habit habit = new Habit(name, motivation, this.nextId);
    this.habits.add(habit);
}

```



```

        int response = nextId;
        this.tracker.put(nextId, new ArrayList<>());
        this.nextId++;
        return response;
    }

    public void addHabitRecord(Integer id) {
        tracker.get(id).add(LocalDate.now());
    }

    public void toggleConcludeHabit(Integer id) {
        for (Habit habit : this.habits) {
            if (habit.getId().equals(id)) {
                habit.setIsConcluded(!habit.getIsConcluded());
            }
        }
    }

    public void removeHabit(Integer id) {
        this.habits.removeIf(habit -> habit.getId().equals(id));
        this.tracker.remove(id);
    }

    public List<LocalDateTime> getHabitRecords(Integer id) {
        return this.tracker.get(id);
    }

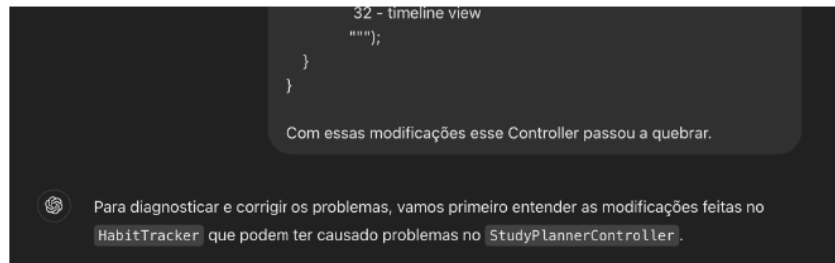
    public List<String> searchInHabits(String search) {
        List<String> habits = new ArrayList<>();
        for (Habit habit : this.habits) {
            if (habit.getName().toLowerCase().contains(search.toLowerCase()) || habit.getMotivation().t
                habits.add(habit.toString());
        }
    }
    return habits;
}

public String getFormattedHabitView() {
    StringBuilder response = new StringBuilder();
    for (Habit habit : habits) {
        response.append("[ Habit: ")
            .append(habit.getName())
            .append(". Records: ");
        List<LocalDateTime> records = getHabitRecords(habit.getId());
        for (LocalDateTime record : records) {
            response.append(formatHabitDate(record)).append(", ");
        }
        response.append("]");
    }
    return response.toString();
}

// Record declared within the class
public record HabitParameters(
    String name,
    String motivation,
    Integer dailyMinutesDedication,
    Integer dailyHoursDedication,
    LocalDateTime startDate,
    Boolean isConcluded
) {}
}

```

2. A refatoração da IA causou efeitos colaterais em outros métodos. Foi então solicitado que esses erros fossem resolvidos:



Resultados:

```
package org.example.controllers;

import org.example.studyplanner.*;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;

public class StudyPlannerController {
    private Map<String, Runnable> actions = new HashMap<>();
    private static TodoTracker todoTracker = TodoTracker.getInstance();
    private static HabitTracker habitTracker = HabitTracker.getHabitTracker();
    private static KanbanView kanbanView = new KanbanView(habitTracker, todoTracker);
    private static TimelineView timelineView = new TimelineView();

    public StudyPlannerController() {
        handlePlannerOptions();
        handleTodoMenuOptions();
        handleHabitMenuOptions();
        handleViewMenuOptions();
    }

    private void handlePlannerOptions() {
        actions.put("1", this::handleTodoInput);
        actions.put("2", this::handleHabitInput);
        actions.put("3", this::handleViewInput);
    }

    private void handleTodoMenuOptions() {
        actions.put("11", this::handleAddTodo);
        actions.put("12", this::handleRemoveTodo);
        actions.put("13", this::handleViewTodo);
        actions.put("14", this::handleViewByPriority);
        actions.put("15", this::handleAddTodoExecution);
    }

    private void handleHabitMenuOptions() {
        actions.put("21", this::handleAddHabit);
        actions.put("22", this::handleRemoveHabit);
        actions.put("23", this::handleViewHabits);
    }

    private void handleViewMenuOptions() {
        actions.put("31", this::handleViewKanban);
        actions.put("32", this::handleViewTimeline);
    }

    private String getInput() {
        return MainController.getInput();
    }
}
```

```

private void handleAddTodoExecution() {
    System.out.println("Type todo id to add a new practiced time");
    Integer id = Integer.parseInt(getInput());
    todoTracker.addToDoExecutionTime(id);
}

private String showTodosByPriority(List<ToDo> todos) {
    return todos.toString();
}

private void handleViewByPriority() {
    List<ToDo> todos = todoTracker.sortTodosByPriority();
    System.out.println(showTodosByPriority(todos));
}

public void handleRemoveTodo() {
    System.out.println("Type todo id to remove");
    Integer id = Integer.parseInt(getInput());
    todoTracker.removeToDo(id);
}

public void handleRemoveHabit() {
    System.out.println("Type habit id to remove");
    Integer id = Integer.parseInt(getInput());
    habitTracker.removeHabit(id);
}

private void handleAddTodo() {
    System.out.println("Type the todo: title, description, priority (number)");
    String title = Objects.requireNonNull(this.getInput().trim());
    String description = Objects.requireNonNull(this.getInput().trim());
    Integer priority = Integer.valueOf(this.getInput());
    todoTracker.addToDo(title, description, priority);
}

private LocalDateTime handleGetStartDate() {
    System.out.println("Enter start date (year, month, day, hour, minute, seconds):");
    int year = Integer.parseInt(getInput());
    int month = Integer.parseInt(getInput());
    int day = Integer.parseInt(getInput());
    int hour = Integer.parseInt(getInput());
    int minute = Integer.parseInt(getInput());
    int seconds = Integer.parseInt(getInput());
    return LocalDateTime.of(year, month, day, hour, minute, seconds);
}

private void handleAddHabit() {
    System.out.println("Separate the input with enter: name, motivation, daily Minutes Dedication,
    String name = Objects.requireNonNull(this.getInput().trim());
    String motivation = Objects.requireNonNull(this.getInput().trim());
    Integer dailyMinutesDedication = Integer.parseInt(Objects.requireNonNull(this.getInput().trim()));
    Integer dailyHoursDedication = Integer.parseInt(Objects.requireNonNull(this.getInput().trim()));
    LocalDateTime startDate = handleGetStartDate();

    // Use HabitParameters record
    HabitTracker.HabitParameters params = new HabitTracker.HabitParameters(
        name,
        motivation,
        dailyMinutesDedication,
        dailyHoursDedication,
        startDate,
        false
    );
}

```

```

        habitTracker.addHabit(params);
    }

    private String viewToDoHeader() {
        return "Todos and latest usages:";
    }

    private void handleViewTodo() {
        System.out.println(viewToDoHeader());
        System.out.println(todoTracker.toString());
    }

    private String viewHabitHeader() {
        return "Habits found: ";
    }

    private void handleViewHabits() {
        System.out.println(viewHabitHeader());
        System.out.println(habitTracker.toString());
    }

    private void handleTodoInput() {
        try {
            while (true) {
                todoOptions();
                String response = MainController.validateInput(actions);
                if (response == null) return;
                actions.get(response).run();
            }
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    private void handleHabitInput() {
        try {
            while (true) {
                habitOptions();
                String response = MainController.validateInput(actions);
                if (response == null) return;
                actions.get(response).run();
            }
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    private void handleViewInput() {
        try {
            while (true) {
                viewOptions();
                String response = MainController.validateInput(actions);
                if (response == null) return;
                actions.get(response).run();
            }
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    private void handleMethodHeader(String header) {
        System.out.println("----" + header + "----");
    }

```

```

private void handleViewKanban() {
    try {
        handleMethodHeader("Kanban view: ");
        System.out.println(kanbanView.kanbanView());
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}

private void handleViewTimeline() {
    try {
        handleMethodHeader("Timeline view: ");
        System.out.println(timelineView.habitDateViewAll(habitTracker));
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}

public void handlePlannerInput() {
    try {
        while (true) {
            controllerOptions();
            String response = MainController.validateInput(actions);
            if (response == null) return;
            actions.get(response).run();
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}

public static void controllerOptions() {
    System.out.println("""
        0 - return
        1 - todo menu
        2 - habit menu
        3 - view menu
        """);
}

public static void todoOptions() {
    System.out.println("""
        0 - return
        11 - add todo
        12 - remove todo
        13 - view todos
        14 - view by priority
        15 - add todo execution date (now)
        """);
}

public static void habitOptions() {
    System.out.println("""
        0 - return
        21 - add habit
        22 - remove habit
        23 - view habit
        """);
}

public static void viewOptions() {
    System.out.println("""
        0 - return
        31 - kanban view
    """);
}

```

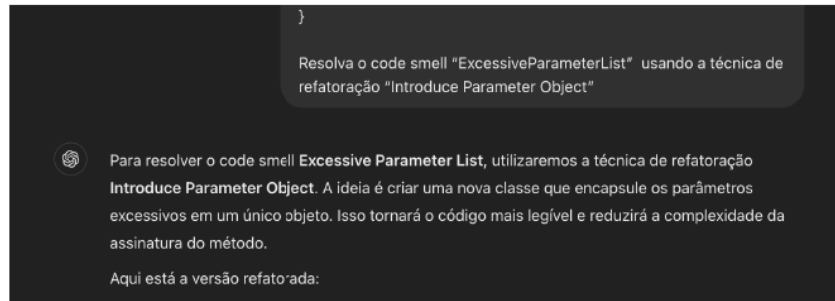
```
        32 - timeline view  
        """);  
    }  
}
```

3. Código livre do code smell e funcionando

ExcessiveParameterList StudyPlan assignSteps

StudyPlan assignSteps

1. Foi enviado para a IA o código da classe StudyPlan com o método assignSteps com o seguinte comando:



Resultado:

```
package org.example.studyregistry;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

// Classe que encapsula os parâmetros
class StepDetails {
    private String firstStep;
    private String resetStudyMechanism;
    private String consistentStep;
    private String seasonalSteps;
    private String basicSteps;
    private String mainObjectiveTitle;
    private String mainGoalTitle;
    private String mainMaterialTopic;
    private String mainTask;
    private Integer numberOfSteps;
    private boolean isImportant;
    private LocalDateTime startDate;
    private LocalDateTime endDate;

    public StepDetails(String firstStep, String resetStudyMechanism, String consistentStep, String seasonalSteps,
                       String basicSteps, String mainObjectiveTitle, String mainGoalTitle, String mainMaterialTopic,
                       String mainTask, Integer numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime endDate) {
        this.firstStep = firstStep;
        this.resetStudyMechanism = resetStudyMechanism;
        this.consistentStep = consistentStep;
        this.seasonalSteps = seasonalSteps;
        this.basicSteps = basicSteps;
        this.mainObjectiveTitle = mainObjectiveTitle;
        this.mainGoalTitle = mainGoalTitle;
        this.mainMaterialTopic = mainMaterialTopic;
        this.mainTask = mainTask;
        this.numberOfSteps = numberOfSteps;
        this.isImportant = isImportant;
        this.startDate = startDate;
        this.endDate = endDate;
    }
}
```

```

// Getters
public String getFirstStep() { return firstStep; }
public String getResetStudyMechanism() { return resetStudyMechanism; }
public String getConsistentStep() { return consistentStep; }
public String getSeasonalSteps() { return seasonalSteps; }
public String getBasicSteps() { return basicSteps; }
public String getMainObjectiveTitle() { return mainObjectiveTitle; }
public String getMainGoalTitle() { return mainGoalTitle; }
public String getMainMaterialTopic() { return mainMaterialTopic; }
public String getMainTask() { return mainTask; }
public Integer getNumberOfSteps() { return numberOfSteps; }
public boolean isImportant() { return isImportant; }
public LocalDateTime getStartDate() { return startDate; }
public LocalDateTime getEndDate() { return endDate; }
}

public class StudyPlan extends Registry {
    private StudyObjective objective;
    private List<String> steps;

    public StudyPlan(String planName, StudyObjective objective, List<StudyMaterial> materials) {
        this.name = planName;
        this.objective = objective;
        this.steps = new ArrayList<>();
    }

    @Override
    public String toString() {
        return "Plan: " + name + ",\nObjective: " + objective.getDescription() + ",\nSteps: " + String
        g.join(", ", steps);
    }

    public List<String> getSteps() {
        return steps;
    }

    public StudyObjective getObjective() {
        return objective;
    }

    public void assignObjective(StudyObjective objective) {
        this.objective = objective;
    }

    public void addSingleStep(String toAdd) {
        steps.add(toAdd);
    }

    // Método refatorado para usar StepDetails
    public void assignSteps(StepDetails stepDetails) {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

        this.steps = new ArrayList<>(Arrays.asList(
            stepDetails.getFirstStep(),
            stepDetails.getResetStudyMechanism(),
            stepDetails.getConsistentStep(),
            stepDetails.getSeasonalSteps(),
            stepDetails.getBasicSteps(),
            "Number of steps: " + stepDetails.getNumberOfSteps(),
            "Is it important to you? " + stepDetails.isImportant(),
            stepDetails.getStartDate().format(formatter),
            stepDetails.getEndDate().format(formatter),
            stepDetails.getMainObjectiveTitle(),

```



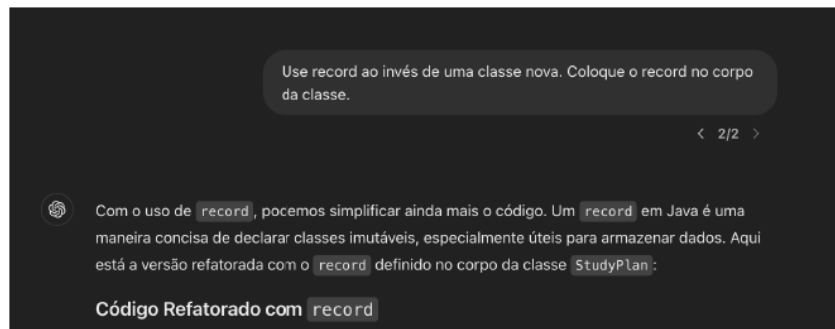
```

        stepDetails.getMainGoalTitle(),
        stepDetails.getMainMaterialTopic(),
        stepDetails.getMainTask()
    ));
}

// Método handleAssignSteps também refatorado
public void handleAssignSteps(List<String> stringProperties, Integer numberOfSteps, boolean isImportant,
                             LocalDateTime startDate, LocalDateTime endDate) {
    StepDetails stepDetails = new StepDetails(
        stringProperties.get(0), stringProperties.get(1), stringProperties.get(2),
        stringProperties.get(3), stringProperties.get(4), stringProperties.get(5),
        stringProperties.get(6), stringProperties.get(7), stringProperties.get(8),
        numberOfSteps, isImportant, startDate, endDate
    );
    assignSteps(stepDetails);
}
}

```

2. Logo em seguida foi pedido que usasse record no lugar de uma nova classe estruturada, para que assim se evitasse alguns efeitos colaterais.



Resultado:

```

package org.example.studyregistry;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class StudyPlan extends Registry {
    private StudyObjective objective;
    private List<String> steps;

    public StudyPlan(String planName, StudyObjective objective, List<StudyMaterial> materials) {
        this.name = planName;
        this.objective = objective;
        this.steps = new ArrayList<>();
    }

    @Override
    public String toString() {
        return "Plan: " + name + ",\nObjective: " + objective.getDescription() + ",\nSteps: " + String.

    public List<String> getSteps() {
        return steps;
    }
}

```

```

public StudyObjective getObjective() {
    return objective;
}

public void assignObjective(StudyObjective objective) {
    this.objective = objective;
}

public void addSingleStep(String toAdd) {
    steps.add(toAdd);
}

// Record para encapsular os detalhes das etapas
public record StepDetails(
    String firstStep,
    String resetStudyMechanism,
    String consistentStep,
    String seasonalSteps,
    String basicSteps,
    String mainObjectiveTitle,
    String mainGoalTitle,
    String mainMaterialTopic,
    String mainTask,
    Integer numberOfSteps,
    boolean isImportant,
    LocalDateTime startDate,
    LocalDateTime endDate
) {}

// Método refatorado para usar StepDetails
public void assignSteps(StepDetails stepDetails) {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

    this.steps = new ArrayList<>(Arrays.asList(
        stepDetails.firstStep(),
        stepDetails.resetStudyMechanism(),
        stepDetails.consistentStep(),
        stepDetails.seasonalSteps(),
        stepDetails.basicSteps(),
        "Number of steps: " + stepDetails.numberOfSteps(),
        "Is it important to you? " + stepDetails.isImportant(),
        stepDetails.startDate().format(formatter),
        stepDetails.endDate().format(formatter),
        stepDetails.mainObjectiveTitle(),
        stepDetails.mainGoalTitle(),
        stepDetails.mainMaterialTopic(),
        stepDetails.mainTask()
    ));
}

// Método handleAssignSteps também refatorado
public void handleAssignSteps(List<String> stringProperties, Integer numberOfSteps, boolean isImport
    LocalDateTime startDate, LocalDateTime endDate) {
    StepDetails stepDetails = new StepDetails(
        stringProperties.get(0), stringProperties.get(1), stringProperties.get(2),
        stringProperties.get(3), stringProperties.get(4), stringProperties.get(5),
        stringProperties.get(6), stringProperties.get(7), stringProperties.get(8),
        numberOfSteps, isImportant, startDate, endDate
    );
    assignSteps(stepDetails);
}
}

```

3. A refatoração da IA causou efeitos colaterais em outros métodos. Foi então solicitado que esses erros fossem resolvidos:

Após a refatoração o controller quebrou. Resolva.



O problema após a refatoração provavelmente está relacionado à interação entre as classes ou métodos que utilizam os novos `record` (`BasicObjectiveDetails` e `AdditionalObjectiveDetails`) e como os dados são coletados e manipulados. Vamos revisar os pontos principais que podem estar causando os problemas e corrigi-los.

Resultados:

```
private StudyObjective.ObjectiveDetails collectObjectiveDetails() {
    try {
        // Coletar detalhes básicos
        var basicDetails = collectBasicObjectiveDetails();
        // Coletar detalhes adicionais
        var additionalDetails = collectAdditionalObjectiveDetails();

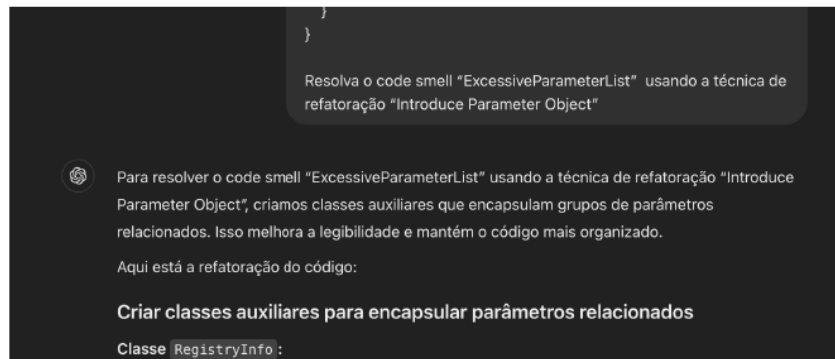
        // Combinar os detalhes em um único registro ObjectiveDetails
        return new StudyObjective.ObjectiveDetails(
            basicDetails.id(),
            basicDetails.priority(),
            basicDetails.practicedDays(),
            basicDetails.targetDate(),
            additionalDetails.name(),
            additionalDetails.title(),
            additionalDetails.description(),
            additionalDetails.topic(),
            additionalDetails.objectiveInOneLine(),
            additionalDetails.objectiveFullDescription(),
            additionalDetails.motivation(),
            additionalDetails.duration(),
            additionalDetails.isActive()
        );
    } catch (Exception e) {
        System.out.println("Error collecting objective details: " + e.getMessage());
        return null;
    }
}
```

4. Código livre do code smell e funcionando

ExcessiveParameterList StudyObjective handleSetObjective

StudyObjective handleSetObjective

1. Foi enviado para a IA o código da classe StudyObjective com o método handleSetObjective com o seguinte comando:



Resultado:

```
package org.example.studyregistry;

import java.time.LocalDateTime;

public class ObjectiveDetails {
    private Integer id;
    private Integer priority;
    private Integer practicedDays;
    private LocalDateTime startDate;
    private String name;
    private String title;
    private String description;
    private String topic;
    private String objectiveInOneLine;
    private String objectiveFullDescription;
    private String motivation;
    private Double duration;
    private boolean isActive;

    public ObjectiveDetails(Integer id, Integer priority, Integer practicedDays, int day, int month,
int year, String name,
                        String title, String description, String topic, String objectiveInOneLin
e,
                        String objectiveFullDescription, String motivation, Double duration, bool
ean isActive) {
        this.id = id;
        this.priority = priority;
        this.practicedDays = practicedDays;
        this.startDate = LocalDateTime.of(year, month, day, 0, 0);
        this.name = name;
        this.title = title;
        this.description = description;
        this.topic = topic;
        this.objectiveInOneLine = objectiveInOneLine;
        this.objectiveFullDescription = objectiveFullDescription;
        this.motivation = motivation;
        this.duration = duration;
        this.isActive = isActive;
    }
}
```

```

    public Integer getId() {
        return id;
    }

    public Integer getPriority() {
        return priority;
    }

    public Integer getPracticedDays() {
        return practicedDays;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }

    public String getName() {
        return name;
    }

    public String getTitle() {
        return title;
    }

    public String getDescription() {
        return description;
    }

    public String getTopic() {
        return topic;
    }

    public String getObjectiveInOneLine() {
        return objectiveInOneLine;
    }

    public String getObjectiveFullDescription() {
        return objectiveFullDescription;
    }

    public String getMotivation() {
        return motivation;
    }

    public Double getDuration() {
        return duration;
    }

    public boolean isActive() {
        return isActive;
    }
}

```

2. Logo em seguida foi pedido que usasse record no lugar de uma nova classe estruturada, para que assim se evitasse alguns efeitos colaterais.

Use record ao invés de uma classe nova

Com o uso do `record`, podemos simplificar ainda mais a refatoração, já que `record` é ideal para representar objetos imutáveis e com apenas dados. Aqui está como ficaria a refatoração usando `record`:

Criar um `record` para encapsular os parâmetros

O `record` substitui a classe auxiliar e reduz o código significativamente:

Resultado:

```
package org.example.studyregistry;

import java.time.LocalDateTime;
import java.util.List;

public class StudyObjective extends Registry {
    private String title;
    private String description;
    private String topic;
    private Integer practicedDays;
    private LocalDateTime startDate;
    private Double duration;
    private String objectiveInOneLine;
    private String objectiveFullDescription;
    private String motivation;

    // Record declarado dentro da classe
    public record ObjectiveDetails(
        Integer id,
        Integer priority,
        Integer practicedDays,
        LocalDateTime startDate,
        String name,
        String title,
        String description,
        String topic,
        String objectiveInOneLine,
        String objectiveFullDescription,
        String motivation,
        Double duration,
        boolean isActive
    ) {}

    public String getTitle() {
        return title;
    }

    public String getTopic() {
        return topic;
    }

    public Integer getPracticedDays() {
        return practicedDays;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }
}
```

```

public Double getDuration() {
    return duration;
}

public String getObjectiveInOneLine() {
    return objectiveInOneLine;
}

public String getObjectiveFullDescription() {
    return objectiveFullDescription;
}

public String getMotivation() {
    return motivation;
}

@Override
public String toString() {
    return "StudyObjective [title:" + title + ", description:" + description
        + (topic != null ? ", topic:" + topic : "")
        + (practicedDays != null ? ", practicedDays:" + practicedDays : "")
        + (duration != null ? ", duration:" + duration : "")
        + (objectiveInOneLine != null ? ", objective summary:" + objectiveInOneLine : "")
        + (objectiveFullDescription != null ? ", objective full description:" + objectiveFullDe
        + (motivation != null ? ", motivation:" + motivation : "") + "]";
}

public StudyObjective(String title, String description) {
    this.title = title;
    this.description = description;
    this.name = title;
}

public void handleSetRegistry(Integer id, String name, Integer priority, boolean isActive) {
    this.id = id;
    this.name = name;
    this.priority = priority;
    this.isActive = isActive;
}

public void handleSetTextualInfo(String title, String description, String topic, String objectiveIn
    String objectiveFullDescription, String motivation) {
    this.title = title;
    this.description = description;
    this.topic = topic;
    this.objectiveInOneLine = objectiveInOneLine;
    this.objectiveFullDescription = objectiveFullDescription;
    this.motivation = motivation;
}

public void handleSetTime(Integer practicedDays, int day, int month, int year, Double duration) {
    this.practicedDays = practicedDays;
    this.duration = duration;
    this.startDate = LocalDateTime.of(year, month, day, 0, 0);
}

public void handleSetObjective(ObjectiveDetails objectiveDetails) {
    this.id = objectiveDetails.id();
    this.priority = objectiveDetails.priority();
    this.practicedDays = objectiveDetails.practicedDays();
    this.startDate = objectiveDetails.startDate();
    this.name = objectiveDetails.name();
    this.title = objectiveDetails.title();
    this.description = objectiveDetails.description();
}

```

```

        this.topic = objectiveDetails.topic();
        this.objectiveInOneLine = objectiveDetails.objectiveInOneLine();
        this.objectiveFullDescription = objectiveDetails.objectiveFullDescription();
        this.motivation = objectiveDetails.motivation();
        this.duration = objectiveDetails.duration();
        this.isActive = objectiveDetails.isActive();
    }

    public int handleSetObjectiveAdapter(List<Integer> intProperties, List<String> stringProperties, Double
        ObjectiveDetails details = new ObjectiveDetails(
            intProperties.get(0),
            intProperties.get(1),
            intProperties.get(2),
            LocalDateTime.of(intProperties.get(3), intProperties.get(4), intProperties.get(5), 0, 0),
            stringProperties.get(0),
            stringProperties.get(1),
            stringProperties.get(2),
            stringProperties.get(3),
            stringProperties.get(4),
            stringProperties.get(5),
            stringProperties.get(6),
            duration,
            isActive
        );
        handleSetObjective(details);
        return details.id();
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }
}

```

3. Foi necessário pedir que refatorasse mais uma vez a classe pois não passou nos testes:

Esse é o erro que dá no teste:
 java.time.DateTimeException: Invalid value for DayOfMonth (valid values 1 - 28/31): 2024

O erro indica que há um problema com os valores de dia, mês ou ano ao criar um objeto `LocalDateTime`. Especificamente, o valor `2024` está sendo usado como o dia do mês, o que é inválido.

O problema está no método `handleSetObjectiveAdapter`, onde estamos extraindo os valores de dia, mês e ano da lista `intProperties`:

```
java
LocalDateTime.of(intProperties.get(3), intProperties.get(4), intProperties.get(5),
```

Os índices das listas `intProperties` não correspondem corretamente a ano, mês e dia. Parece que os valores para dia, mês e ano estão fora de ordem.

Resultado:

```

public int handleSetObjectiveAdapter(List<Integer> intProperties, List<String> stringProperties, Double
    ObjectiveDetails details = new ObjectiveDetails(
        intProperties.get(0), // id
        intProperties.get(1), // priority

```

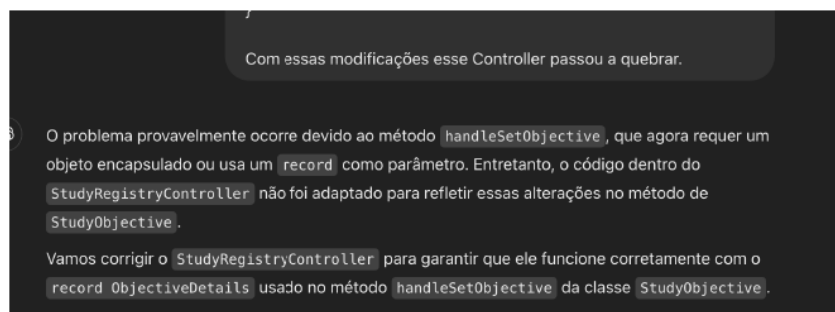


```

        intProperties.get(2), // practicedDays
        LocalDateTime.of(
            intProperties.get(5), // year
            intProperties.get(4), // month
            intProperties.get(3), // day
            0, 0
        ),
        stringProperties.get(0), // name
        stringProperties.get(1), // title
        stringProperties.get(2), // description
        stringProperties.get(3), // topic
        stringProperties.get(4), // objectiveInOneLine
        stringProperties.get(5), // objectiveFullDescription
        stringProperties.get(6), // motivation
        duration,
        isActive
    );
    handleSetObjective(details);
    return details.id();
}

```

4. A refatoração da IA causou efeitos colaterais em outros métodos. Foi então solicitado que esses erros fossem resolvidos:



Resultados:

```

private void handleSetObjective(StudyObjective objective) {
    handleMethodHeader("(Study Objective Edit)");
    System.out.println("Type the following info: Integer id, Integer priority, Integer practicedDays, i
    int id = Integer.parseInt(getInput());
    int priority = Integer.parseInt(getInput());
    int practicedDays = Integer.parseInt(getInput());
    int day = Integer.parseInt(getInput());
    int month = Integer.parseInt(getInput());
    int year = Integer.parseInt(getInput());
    String name = getInput();
    String title = getInput();
    String description = getInput();
    String topic = getInput();
    String objectiveInOneLine = getInput();
    String objectiveFullDescription = getInput();
    String motivation = getInput();
    double duration = Double.parseDouble(getInput());
    boolean isActive = Boolean.parseBoolean(getInput());

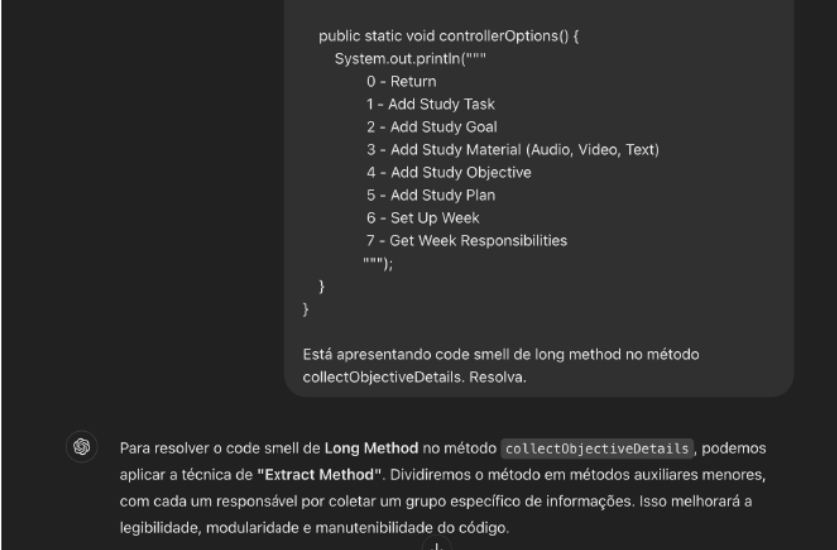
    // Criando o ObjectiveDetails
    StudyObjective.ObjectiveDetails details = new StudyObjective.ObjectiveDetails(
        id, priority, practicedDays, LocalDateTime.of(year, month, day, 0, 0), name, title, description
        objectiveInOneLine, objectiveFullDescription, motivation, duration, isActive
    );

    // Passando para o método handleSetObjective
}

```

```
objective.handleSetObjective(details);
}
```

5. Alguns code smells resultantes tiveram que ser resolvidos



```
public static void controllerOptions() {
    System.out.println("""
        0 - Return
        1 - Add Study Task
        2 - Add Study Goal
        3 - Add Study Material (Audio, Video, Text)
        4 - Add Study Objective
        5 - Add Study Plan
        6 - Set Up Week
        7 - Get Week Responsibilities
        """);
}
```

Está apresentando code smell de long method no método collectObjectiveDetails. Resolva.

Para resolver o code smell de **Long Method** no método `collectObjectiveDetails`, podemos aplicar a técnica de **"Extract Method"**. Dividiremos o método em métodos auxiliares menores, com cada um responsável por coletar um grupo específico de informações. Isso melhorará a legibilidade, modularidade e manutenibilidade do código.

```
private StudyObjective.ObjectiveDetails collectObjectiveDetails() {
    int id = collectIdAndPriority("ID");
    int priority = collectIdAndPriority("priority");
    int practicedDays = collectIdAndPriority("practiced days");
    LocalDateTime startDate = collectStartDate();
    String name = collectTextualInfo("name");
    String title = collectTextualInfo("title");
    String description = collectTextualInfo("description");
    String topic = collectTextualInfo("topic");
    String objectiveInOneLine = collectTextualInfo("objective in one line");
    String objectiveFullDescription = collectTextualInfo("full description");
    String motivation = collectTextualInfo("motivation");
    double duration = collectNumericInfo("duration");
    boolean isActive = collectBooleanInfo();

    return new StudyObjective.ObjectiveDetails(
        id, priority, practicedDays, startDate, name, title, description, topic,
        objectiveInOneLine, objectiveFullDescription, motivation, duration, isActive
    );
}

private int collectIdAndPriority(String fieldName) {
    return promptInt("Type the " + fieldName + ": ");
}

private LocalDateTime collectStartDate() {
    System.out.println("Type the start date:");
    int day = promptInt("Day: ");
    int month = promptInt("Month: ");
    int year = promptInt("Year: ");
    return LocalDateTime.of(year, month, day, 0, 0);
}

private String collectTextualInfo(String fieldName) {
    return promptString("Type the " + fieldName + ": ");
}

private double collectNumericInfo(String fieldName) {
```

```
        return promptDouble("Type the " + fieldName + ": ");
    }

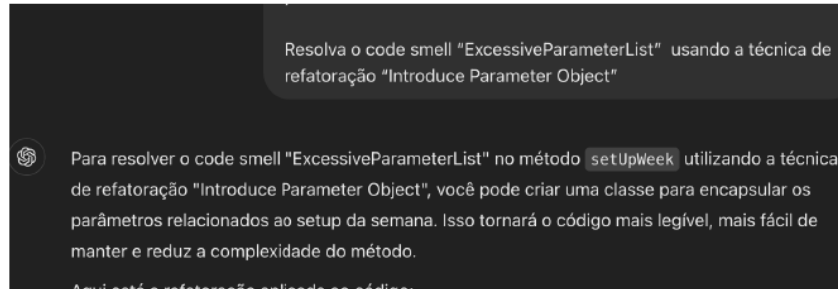
    private boolean collectBooleanInfo() {
        return promptBoolean("Is the objective active? (true/false): ");
    }
}
```

6. Código livre do code smell e funcionando

ExcessiveParameterList StudyTaskManager setUpWeek

StudyTaskManager setUpWeek

1. Foi enviado para a IA o código da classe StudyTaskManager com o método setUpWeek com o seguinte comando:



Resultado:

```
package org.example.studyregistry;

import java.util.Objects;

public class WeekSetup {
    private String planName;
    private String objectiveTitle;
    private String objectiveDescription;
    private String materialTopic;
    private String materialFormat;
    private String goal;
    private String reminderTitle;
    private String reminderDescription;
    private String mainTaskTitle;
    private String mainHabit;
    private String mainCardStudy;

    public WeekSetup(String planName, String objectiveTitle, String objectiveDescription, String materialTopic,
                     String materialFormat, String goal, String reminderTitle, String reminderDescription,
                     String mainTaskTitle, String mainHabit, String mainCardStudy) {
        this.planName = planName;
        this.objectiveTitle = objectiveTitle;
        this.objectiveDescription = objectiveDescription;
        this.materialTopic = materialTopic;
        this.materialFormat = materialFormat;
        this.goal = goal;
        this.reminderTitle = reminderTitle;
        this.reminderDescription = reminderDescription;
        this.mainTaskTitle = mainTaskTitle;
        this.mainHabit = mainHabit;
        this.mainCardStudy = mainCardStudy;
    }

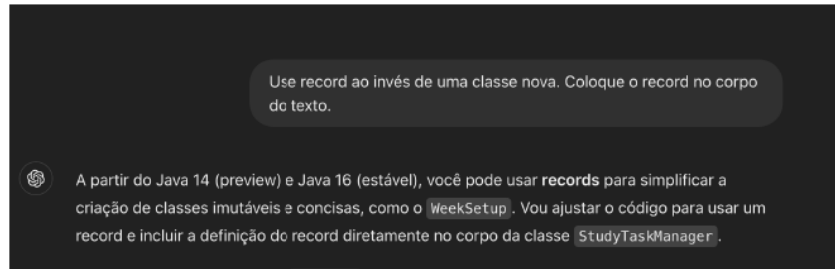
    // Getters and setters
    public String getPlanName() { return planName; }
    public String getObjectiveTitle() { return objectiveTitle; }
    public String getObjectiveDescription() { return objectiveDescription; }
    public String getMaterialTopic() { return materialTopic; }
    public String getMaterialFormat() { return materialFormat; }
    public String getGoal() { return goal; }
```

```

    public String getReminderTitle() { return reminderTitle; }
    public String getReminderDescription() { return reminderDescription; }
    public String getMainTaskTitle() { return mainTaskTitle; }
    public String getMainHabit() { return mainHabit; }
    public String getMainCardStudy() { return mainCardStudy; }
}

```

2. Logo em seguida foi pedido que usasse record no lugar de uma nova classe estruturada, para que assim se evitasse alguns efeitos colaterais.



Resultado:

```

package org.example.studyregistry;

import java.util.ArrayList;
import java.util.List;

public class StudyTaskManager {
    private static StudyTaskManager instance;
    private StudyMaterial studyMaterial = StudyMaterial.getStudyMaterial();
    List<Registry> registryList;
    List<String> weekResponsibilities = List.of();

    // Record para encapsular os dados do setup da semana
    public record WeekSetup(
        String planName,
        String objectiveTitle,
        String objectiveDescription,
        String materialTopic,
        String materialFormat,
        String goal,
        String reminderTitle,
        String reminderDescription,
        String mainTaskTitle,
        String mainHabit,
        String mainCardStudy
    ) {}

    private StudyTaskManager() {
        this.registryList = new ArrayList<>();
    }

    public static StudyTaskManager getStudyTaskManager() {
        if (instance == null) {
            instance = new StudyTaskManager();
        }
        return instance;
    }

    public List<String> getWeekResponsibilities() {
        return weekResponsibilities;
    }

    public void setUpWeek(WeekSetup weekSetup) {

```

```

        this.weekResponsibilities = new ArrayList<>();
        this.weekResponsibilities.addAll(List.of(
            weekSetup.planName(),
            weekSetup.objectiveTitle(),
            weekSetup.objectiveDescription(),
            weekSetup.materialTopic(),
            weekSetup.materialFormat(),
            weekSetup.goal(),
            weekSetup.reminderTitle(),
            weekSetup.reminderDescription(),
            weekSetup.mainTaskTitle(),
            weekSetup.mainHabit(),
            weekSetup.mainCardStudy()
        ));
    }

    public void handleSetUpWeek(List<String> stringProperties) {
        if (stringProperties.size() == 11) {
            WeekSetup weekSetup = new WeekSetup(
                stringProperties.get(0),
                stringProperties.get(1),
                stringProperties.get(2),
                stringProperties.get(3),
                stringProperties.get(4),
                stringProperties.get(5),
                stringProperties.get(6),
                stringProperties.get(7),
                stringProperties.get(8),
                stringProperties.get(9),
                stringProperties.get(10)
            );
            setUpWeek(weekSetup);
        }
    }

    public void addRegistry(Registry registry) {
        registryList.add(registry);
    }

    public void removeRegistry(Registry registry) {
        registryList.remove(registry);
    }

    public List<Registry> getRegistryList() {
        return registryList;
    }

    public List<String> searchInRegistries(String text) {
        List<String> response = new ArrayList<>();
        for (Registry registry : registryList) {
            String mix = (registry.getName() != null ? registry.getName() : "");
            if (mix.toLowerCase().contains(text.toLowerCase())) {
                response.add(registry.getName());
            }
        }
        return response;
    }
}

```

3. A refatoração da IA causou efeitos colaterais em outros métodos. Foi então solicitado que esses erros fossem resolvidos:

Após a refatoração esse método quebrou:

```
private void getWeekInfo() {
    System.out
        .println("(Study Task Manager Week Set Up) Type the
following info: String planName, String objectiveTitle, String
objectiveDescription, String materialTopic, String materialFormat,
String goal, String reminderTitle, String reminderDescription, String
mainTaskTitle, String mainHabit, String mainCardStudy");
    studyTaskManager.setUpWeek(getInput(), getInput(),
getInput(), getInput(), getInput(), getInput(),
getInput(), getInput(), getInput());
}
```

< 2/2 >



Após a refatoração, o método `setUpWeek` passou a aceitar um objeto do tipo `WeekSetup` como argumento, e não uma lista de parâmetros individuais. Para corrigir o método `getWeekInfo`, você deve criar um objeto `WeekSetup` utilizando os valores obtidos pelo método `getInput` e, em seguida, passá-lo ao método `setUpWeek`.

Aqui está o código corrigido:

Resultados:

```
private void getWeekInfo() {
    System.out.println("(Study Task Manager Week Set Up) Type the following info: ");
    System.out.println("String planName, String objectiveTitle, String objectiveDescription, String mat

    // Coleta os inputs para cada campo
    String planName = getInput();
    String objectiveTitle = getInput();
    String objectiveDescription = getInput();
    String materialTopic = getInput();
    String materialFormat = getInput();
    String goal = getInput();
    String reminderTitle = getInput();
    String reminderDescription = getInput();
    String mainTaskTitle = getInput();
    String mainHabit = getInput();
    String mainCardStudy = getInput();

    // Cria um objeto WeekSetup com os dados coletados
    StudyTaskManager.WeekSetup weekSetup = new StudyTaskManager.WeekSetup(
        planName,
        objectiveTitle,
        objectiveDescription,
        materialTopic,
        materialFormat,
        goal,
        reminderTitle,
        reminderDescription,
        mainTaskTitle,
        mainHabit,
        mainCardStudy
    );

    // Passa o objeto para o método setUpWeek
    studyTaskManager.setUpWeek(weekSetup);
}
```

4. Foi solicitado em seguida que um code smell resultante deste também fosse resolvido:



Após a refatoração, esse método começou a afirmar o code smell de Long Method, resolva usando Extract Method:

```
private void getWeekInfo() {  
    System.out.println("(Study Task Manager Week Set Up) Type  
the following info: ");
```

5. Código livre do code smell e funcionando